Forma App Platform Product Definition

Platform purpose capabilities experience and launch scope

Version 1.0 | October 22, 2026

Forma is a cloud platform where teams turn their data and intent into working apps, then run, adapt, and share those apps in their company workspace. Forma is the product from the first release. It provides the creation environment, reusable components, hosting, access, saved work, and application lifecycle. Teams use those capabilities to create the tools their work requires.

The app becomes both the thing people share and the place they do the work. A spreadsheet can be an input. A conversation can express intent. The durable result is a working application with an owner, a known version, appropriate access, and saved work that another person can understand.

Apps are what teams create with Forma. A security investigation, a finance review, and an operations tracker are examples of those outputs. Northstar is a reference demonstration of the platform. Company-defined spaces organize the apps: typing /security or clicking Security opens the same space, while another space can contain different apps built through the same product.

This document defines the platform to build, its users, capabilities, experience, scope, positioning, and remaining decisions. It is the current product definition. The earlier security document and original demonstrations remain reference examples of what can be created with Forma. Their domain-specific requirements do not determine the platform's launch market or identity.

Decision status: Confirmed direction comes from the conversation. Adopted defaults resolve ordinary product choices so the team can proceed. Open decisions appear in the final section with a recommended default and the evidence needed to resolve them.

Implementation status: The capabilities below describe the intended product. Forma Stack 3.0.0 provides relevant foundations; its existence does not establish that the complete hosted platform is implemented. Section 15 separates the foundation, prototypes, and product requirements.

Contents

1 Product purpose and the customer promise
2 Conversation decisions and adopted defaults
3 Users buyers and the first job
4 The product concepts and their relationships
5 Space navigation and visual identity
6 Home app discovery and starting work
7 The canvas and everyday interaction
8 Data import connections and interpretation
9 Generative apps reusable modules and precise edits
10 Workflows decisions and external actions
11 Collaboration sharing and independent reuse
12 Publication ownership and application lifecycle
13 Secure cloud organization access and data handling
14 AI APIs and MCP in the product
15 Why Forma Stack matters and what is established
16 The platform story and six act demonstration
17 Example apps and the creator platform analogy
18 Release scope and product completeness
19 Adoption measurement and the commercial approach
20 Positioning the YC connection and the explanation to use
21 Reference notes
22 Remaining open decisions

1 Product purpose and the customer promise

Forma gives a company a home for the small applications its teams need. These apps solve defined jobs: investigating an alert, reviewing exceptions, reconciling records, coordinating a decision, or following a recurring operational process. The team should be able to create a useful tool, use it with real data, and make it available to colleagues without each app becoming a separate deployment project.

Layer	Role in the product
Forma Stack	The technical foundation used to build and run the platform experience.
Forma platform	The product customers adopt to create, host, use, adapt, and share their apps.
Team-created apps	The tools produced and maintained inside Forma for particular jobs.
Demonstrations	Examples such as Northstar that make the platform's capabilities understandable.

The customer promise covers the life of the app. Forma helps people bring in data, create or adapt the interface and rules, review meaningful changes, publish a version, perform work, and reuse the tool. Hosting, identity, access, saved state, application ownership, and maintenance are part of that promise. A generated screen is one step in delivering it.

The intended change in everyday work is concrete. A colleague can send a link to a reconciliation app with the relevant period, exceptions, decisions, and evidence preserved. Another colleague can continue that work, subject to access, or start a different reconciliation using the reusable app. The team does not have to reconstruct the workflow from a collection of detached files and private prompts.

Forming and transforming apps

Forma forms an app around a team's intent and data. The team can assemble reusable components, generate supported missing behavior, and establish the views, inputs, rules, and interactions needed for a task. The app then becomes a maintained tool in the company workspace.

As the work changes, the team can transform that app: add a comparison, introduce a supported form, adjust a calculation, reorganize the canvas, or change a defined workflow. Material changes remain understandable and reviewable, and saved work retains its relevant context and history.

A company can build a collection of these apps across spaces. This is the meaning of forming and transforming in the product: the tools take shape around the work and can change with it. Space navigation selects the relevant apps and context; generating new behavior is a separate visible operation.

Spreadsheets as an entry point

"Spreadsheets become apps" is a useful way to introduce Forma. Someone drops in a workbook, explains the task, and obtains views and interactions suited to the data. The resulting app can retain the rules, decisions, and work that a raw table does not express on its own.

The platform also accepts supported logs, files, connected databases, and business systems. A source does not have to be a spreadsheet, and connecting a system does not mean copying its entire contents into Forma. The product makes the selected data and its handling understandable.

Apps as the shared place of work

"The app becomes the new shared document" describes the desired experience. People share a useful working environment that includes data views, calculations, decisions, notes, and a defined workflow. They can return to it and understand what happened.

This analogy does not require every document to become an app or imply that simultaneous editing already exists. The first product supports persistent shared work and handoff. Live coediting is a later capability when it improves the actual workflow.

The meaning of end to end

Forma can support a complete, defined digital workflow. A team might collect internal review inputs, compare records, record decisions, and hand off the remaining work. The Northstar demonstration applies the same idea to an investigation. More advanced workflows may change an external system through a specifically supported action.

An API, MCP, AI, and data make many workflows possible. They still need correct business rules, a usable interface, persistent decisions, appropriate authority, and dependable execution. We will describe end-to-end coverage by naming the workflow and its start and finish. The initial platform supports a defined set of app capabilities across work areas. Additional integrations and behaviors expand what teams can build.

2 Conversation decisions and adopted defaults

The conversation explored insurance and security demonstrations before confirming Forma itself as the product to build and launch. Those demonstrations now illustrate apps created on the platform. The security work preserves a useful example of canvas interaction, ingestion, generation, review, and reuse, without defining a mandatory first industry or a separate security product.

The table records the decisions that should remain stable while the separate technical design is developed. Confirmed indicates direction expressed in the conversation. Default indicates a practical choice adopted for this definition.

Decision	Status	Product consequence
Forma is a company app platform	Confirmed	Lead with building, running, and sharing apps over company data.
The app is the shared unit of work	Confirmed	Support ongoing work, saved context, handoff, and reuse.
Teams create and transform their own apps	Confirmed	Apps are outputs of the platform, within its supported capabilities.
Northstar and other scenarios are demonstrations	Confirmed	Use concrete examples to explain Forma without defining its market by the demo.

Decision	Status	Product consequence
One platform serves different work areas	Confirmed	Use company spaces rather than separate industry products.
Slash commands and clicks both work	Confirmed	Both routes open the same space or app.
Space presentation changes promptly	Confirmed	Change relevant apps, title, accent, and context within a familiar shell.
Canvas and file drop remain central	Confirmed	Keep a spatial work surface and accessible upload controls.
Generation works with reusable building blocks	Confirmed	Find or adapt suitable tools and generate supported missing pieces.
Forma Stack 3.0.0 is the foundation	Confirmed	Explain its contribution without equating the framework with the finished platform.
Company app libraries are private by default	Default	Public distribution and cross-company templates come later.
Apps and saved work are distinct	Default	Reusing an app does not copy its author's evidence or authority.
Start with one managed cloud offering	Default	Establish a usable company account and clearly separated customer access.
Initial connected sources are read-only	Default	Support internal forms, records, and decisions; stage external actions explicitly.
Browser workspace comes first	Default	Add a selected assistant host after the core experience works.
Validate the platform through concrete work	Default	Select recurring customer tasks and prove that the same product supports different apps.

The first company account may contain only one active space. That does not change the product model. Spaces can be added as teams acquire useful apps, without presenting empty industry categories as completed products.

What carries forward from Gemini

We retain the canvas, spreadsheet and log import, prepared component pack, reuse before generation, meaningful custom views, precise edits, and the Maya to Sarah to Jordan demonstration. A specific denied operation can make the access controls tangible. Model and MCP integrations remain part of the direction, introduced in stages.

We refine the suggestion that the pivot only changes sample files. Reusable technology can carry forward, but application semantics, data interpretation, permissions, and the customer workflow still need implementation and validation. We also separate a quick presentation edit from a changed calculation.

We exclude unsupported fixed timings, a guaranteed 90 percent reuse ratio, zero service cost, claims that competitors will crash, and claims that data can never be exfiltrated. None of those assertions is needed to explain the product. Actual user outcomes and measured controls should support the pitch.

3 Users buyers and the first job

The initial users are people who create small tools for their teams and the colleagues who use those tools. A builder may be an operations practitioner, technical analyst, or engineer. A user may need a review queue, tracker, comparison, investigation, or another supported app. The common need is a useful team tool that can be created, operated, shared, and adapted in one place.

The likely champion is someone who currently maintains spreadsheets, queries, scripts, dashboards, or recurring reports for a team. The purchasing sponsor is the leader accountable for that team's work and tooling. These are starting hypotheses to validate with actual customers, not evidence of demand.

Role	Main responsibility in Forma	Desired outcome
Practitioner	Uses apps to examine data, make supported judgments, and save work.	Complete a recurring task with less reconstruction.
Builder or technical champion	Creates and adapts apps, establishes rules, and maintains useful tools.	Help colleagues without rebuilding the same workflow repeatedly.
Reviewer or source owner	Checks material logic, data interpretation, access, and publication changes.	Understand what is being approved and why.
Team leader or buyer	Chooses valuable workflows and evaluates adoption and cost.	Repeated operational value worth purchasing.
Organization administrator	Manages membership, spaces, policies, sources, and ownership.	Keep the company's apps usable and appropriately controlled.

One person may hold more than one role. Northstar uses three people to make the story clear; a small team should not need three separate employees merely to operate Forma. Separation of responsibilities follows the organization's policy.

The first job to solve is: help our team form a working app around its data and task, run that app reliably, share the work, and adapt or reuse the tool when the need changes. An early pilot should have usable data, a recurring task, a named owner, and at least two people who can test the collaboration promise.

Pilot selection follows access to real users, repeated need, and willingness to use and pay for the platform. Existing security relationships may help recruit a team, but the demo does not commit Forma to a security-only launch. A different supported app must be possible through the same creation path, without the team waiting for Forma's engineers to build a separate industry product.

4 The product concepts and their relationships

Forma should present a small set of consistent concepts. They describe the experience; the technical design can choose the implementation behind them.

Concept	Meaning	Example
Organization	A company account with its membership, policies, and access boundaries.	The customer's company

Concept	Meaning	Example
Space	A company-defined area containing related apps and work.	Security or Finance
App	A reusable tool for a defined task, including its views, rules, and permitted interactions.	A spend review or investigation app
App version	A particular published definition of an app.	Spend Review version 1
Saved work	A continuing use of an app with its own context, records, notes, and state.	October review or a particular investigation
Canvas	A working surface that arranges related views, evidence, and notes.	A comparison, detail view, form, and notes
Widget or card	A view or control inside an app.	Timeline, table, or input form
Data source or revision	Selected imported or connected information with its source history.	A selected workbook table or log import

Saved work is called a case in the security app. A finance app might call it a reconciliation period; an operations app might present a continuing board. The platform should allow domain-appropriate names without treating all business work as an incident.

An app definition contains what the tool needs and how it behaves. A saved case contains the actual investigation context, selected evidence, notes, and decisions. Publishing the tool and sharing the case are different operations, explained in Section 11.

A space organizes work. Access rules determine what a person can see and do within it. Space membership may participate in those rules, but a space title, theme, or slash command does not itself establish access to every app or source.

Adopted default: An app has one owning space and one maintained identity. Other permitted spaces may link to it without creating an unmanaged copy. A deliberate fork creates a separate app with its own owner and history. Moving an app between spaces must make any audience or policy change visible.

5 Space navigation and visual identity

Forma has one recognizable platform shell. The organization selector, space navigation, app search, and access cues stay in predictable locations. A selected space changes the relevant catalog, title, icon, accent, templates, and shortcuts. Users retain their light or dark preference and other accessibility settings.

The command box offers fast navigation. Clicking remains visible and fully supported so people can discover the same destinations without remembering commands. Both methods resolve to one destination and preserve the same work state.

User action	Destination or behavior
Type /security and press Enter	Open the Security space and its permitted apps.
Click Security in navigation	Open the same Security space.

User action	Destination or behavior
Type /security/outbound and select it	Open the Outbound Investigation app in its owning space.
Click that app in All apps	Open the same app with Security context visible.
Select a timeline widget	Focus, expand, or interact within the current app.
Switch the organization selector	Explicitly move to another company account, if authorized.

Typing a partial command shows matching destinations. Navigation occurs when the person presses Enter or selects a result. An unknown command explains the problem and offers permitted matches. It does not create a space, run an external action, or guess a new organization. Slash navigation belongs in the global command box and does not intercept ordinary typing in notes, log searches, or formula editors.

Routine navigation does not need an AI generation call. The shell can respond promptly while the selected app checks access and loads its data. Loading, unavailable sources, and denied access must remain visible; a fast theme change does not imply that new data is already ready.

Returning to an app restores its saved layout, filters, selected work, and notes where appropriate, while checking current access. Before leaving work, Forma saves pending changes or clearly explains any unsaved content that needs attention. State is kept within the relevant organization and work item. Switching from Security to Finance must not carry a security case's results into the finance view.

Spaces are company defined. Security, Finance, and Operations are useful examples, not a fixed list of industries. Administrators can name spaces and establish unique aliases. If an alias changes, saved links should continue to identify the same app or clearly explain the change. Ambiguous names should show the space and app together.

An empty authorized space gives an honest starting point: create an app, use an available starter, or request the access the organization permits. Search results and command suggestions must not expose private titles, thumbnails, source details, or cached findings to unauthorized people.

6 Home app discovery and starting work

Home answers two questions: what can I work on, and what was I doing? It shows permitted recent work, available apps, relevant spaces, and clear routes to create or upload. An app card explains its purpose, owner, current published version, required inputs, and availability.

The initial catalog is private to the organization. Search supports exact names, ordinary language, and useful filters such as space, task, owner, and publication status. People should be able to find an existing app before asking the system to create another one.

Semantic retrieval helps match intent to apps and components even when the wording differs from their names. The product checks authorization, publication status, required data, and supported behavior before offering a result for use. A similarity score alone is insufficient grounds to run an app or expose its contents.

Starting an app offers an explicit choice between continuing existing work and starting new work. A new instance asks for its purpose and compatible permitted sources. It does not silently open another user's case because the app was originally created there.

Creating an app begins with a task, a starter, or data. Forma explains the available information and proposes a useful first version. A new user should see a working starting arrangement rather than an unexplained blank canvas. Advanced users can start with an empty canvas when that fits their task.

Unpublished drafts belong to their permitted builders and reviewers. Published apps are available to their intended audiences. Retired apps remain understandable in historical work but are not recommended for new work. Apps with missing dependencies show a specific issue instead of an apparently valid empty result.

7 The canvas and everyday interaction

The canvas remains a central Forma capability. It allows a person to examine related views side by side, place reference material near a decision, and preserve a useful working arrangement. Investigations, reviews, and planning apps can all use this surface when it suits their task.

The canvas supports pan and zoom, moving and resizing cards, grouping related content, useful automatic placement, and saved layouts. New content appears near the relevant work without covering it. A reset or fit-to-content control helps a person recover orientation. Keyboard navigation and a structured list or focused view provide an alternative to spatial navigation.

Related cards can share a selection or filter when the app defines that relationship. Selecting a destination can update its timeline, inventory comparison, and evidence table. The interface shows which filters are active and which views they affect. Clearing a filter returns to the prior scope predictably.

Connecting lines and grouping communicate a real relationship: a source feeds a view, or a finding references evidence. Decorative connections should not imply that the app has established a causal relationship. Data origin and a business conclusion remain distinguishable.

The app can also present a focused table, form, review queue, or step-based workflow where that serves the task. The platform should not force every routine interaction to occupy an expansive canvas. These presentations use the same app, data rules, and saved work.

Stable work during change

Ordinary approved interactions respond directly. Expanding details or changing a permitted filter does not require a new application review. When AI proposes a new layout, view, or interaction, the change is visible and can be previewed, accepted, undone, or discarded.

The user's current work should remain available if generation fails or a proposed change is unsuitable. A targeted label edit should preserve unrelated filters and notes. A failed calculation must display an error rather than a zero. A source outage must be distinguishable from a valid result containing no matching records.

The product displays save state clearly. The task context, selected sources, committed records, and decisions are shared with the saved work's authorized audience, subject to source access. A person's exploratory filters and layout remain personal unless deliberately saved as a shared view or included in a handoff. Published app changes have separate consequences. A team should be able to understand which changes other people will see.

8 Data import connections and interpretation

Data entry starts with a visible drop zone and an equivalent Upload control. Dropping a supported file onto the canvas creates a source card and begins inspection. The first release supports CSV and bounded tabular XLSX import. Supported formats, limits, and relevant restrictions are visible before upload.

The import sequence is straightforward: select the source, inspect the relevant table, confirm important field meanings, resolve material problems, and save a usable revision. The user can see progress, cancel, and understand a failure. The app preserves source history so later findings can be traced to the information used.

Spreadsheet behavior

For CSV, the user can confirm headers, delimiters, encodings, quoted values, and missing fields. Errors identify affected rows and explain the impact. Accepted and rejected row counts remain visible; rejected records do not disappear without an explanation.

For XLSX, the user selects worksheets or tables. The first release reads supported stored values and reports formula-related limitations. It does not promise to run workbook macros, external links, or a complete Excel calculation engine. A workbook without usable values should explain the problem and the available remedy.

AI can suggest field mappings and sensitive fields. The user confirms consequential interpretations such as timezones, currency, units, identifiers, and matching keys. The product must not silently treat bytes as megabytes, local times as UTC, or an unknown account field as an approved join key.

Source history and meaningful revisions

An import records its filename, uploader, time, source context, mapping decisions, row counts, and relevant transformations. Replacing a file creates a new revision. Existing findings remain associated with the evidence used at the time rather than quietly changing meaning when new data arrives.

Connected sources need equally clear behavior. An app explains what it reads, how current it is, whether it is a snapshot or refreshable view, and what information Forma retains. A refreshed result should identify its new observation time or revision. Shared work must make it possible to understand which result supported a saved decision.

The first live connection will follow a repeated pilot need. Supported APIs, database connections, and MCP tools are ways to provide selected data or actions. A connector can fail, lose permission, change its schema, or return only part of a request. Those conditions must be visible and must not be presented as successful complete analysis.

Correct analysis over the data

Each app declares the data it needs and the meaning of its calculations. Compatible column names alone do not prove that two datasets represent the same thing. Matching rules, units, time windows, deduplication, and missing data can change the outcome and need an understandable definition.

Sensitivity suggestions help apply the organization's policy, but cannot guarantee that every sensitive value was recognized. Importing evidence does not grant permission to publish or export it. Where the task can be completed with selected fields or aggregates, the app should avoid unnecessary exposure.

Large-file capability is a measured service characteristic. The product should remain responsive through progress, bounded results, filtering, cancellation, and clear supported limits. We will establish file, row, query, and concurrency limits from representative workloads rather than inherit the unverified figures in Gemini's recommendations.

9 Generative apps reusable modules and precise edits

Generative UI means that the working app can be created or adapted to the user's task. A request can change a view, comparison, calculation, interaction, or arrangement. The output is an inspectable application that can be kept and reused.

The product follows three possible paths. It finds a suitable existing app or component; it adapts a compatible one; or it creates a draft for supported missing functionality. Forma should explain which path it took. An unrelated preset should not be presented as successful generation.

The prepared core pack

The first product ships reusable components for summaries, tables, timelines, comparisons, filters, forms, notes, and supported review or status interactions. Teams combine these into different apps. A form can collect or update permitted internal records; a table can inspect them; a comparison can help explain an exception. Northstar configures a subset into its investigation example.

The pack should grow from real tasks. Generation may reuse a table renderer while introducing a new comparison or calculation. The value lies in the new working behavior, not in inventing an unfamiliar visual primitive each time. Reuse is measured across customer work; there is no fixed 90 percent promise.

The role of the vector database

The intended vector capability helps discover relevant apps and modules by meaning, supports finding reusable structures, and can assist targeted adaptation where the application supports it. It is a discovery and reuse capability, not a replacement for access checks or validated calculations.

Exact totals, record matching, and evidence references come from the permitted data operations defined by the app. A semantically similar result does not establish a correct answer. The registry must respect organization boundaries and avoid returning private app content, evidence, or metadata through search.

Creation guidance and reviewable changes

The request interface helps the person specify the task, relevant sources, desired result, and missing information. It asks a focused question when an unresolved interpretation would materially affect the result. Routine choices use visible defaults. Requests outside supported capabilities receive a clear limitation and an actionable next step.

The user can inspect a draft, test it against representative permitted data, and accept or undo proposed changes. Generation has a visible state, including partial completion and failure. A working published app remains available while a new version is being developed.

Precise edits are an important experience. Changing a title should change that title. Changing the layout should preserve the investigation where possible. Changing a measure requires its actual

definition to change. For example, converting total bytes to bytes per minute requires a defined time interval, denominator, and unit; renaming the metric would be incorrect.

Material changes to shared logic, sources, joins, or actions create a draft subject to the publication policy. Personal arrangement and approved filtering remain ordinary use. The interface should make this difference understandable without exposing internal runtime machinery to every user.

10 Workflows decisions and external actions

An app should define what completion means. An investigation ends with a supported handoff or a documented unresolved question. A reconciliation ends with explained matches and exceptions. A review queue records decisions and remaining work. A dashboard that displays numbers without helping a person complete the intended task may be insufficient for the product promise.

The initial platform reads permitted source data and supports creating or updating defined records inside Forma, including form responses, notes, status, ownership, and decisions. This capability boundary lets teams build useful apps across work areas. Writing to a connected system or sending a message externally requires a separately supported and authorized action.

An external action becomes a product capability only when its meaning, authority, and result are defined. A button labeled Approve might record an internal decision, authorize a payment, or update a source record. Those effects must be explicit before the person uses it.

For each supported external action, the app explains what will change, where it will change, who can perform it, and whether approval is required. It shows progress and the observed result. A timeout can leave the outcome uncertain; the interface must not convert uncertainty into an unsupported success claim.

Retry behavior must account for duplicate effects. The connected system's capabilities determine how an action can be retried or reconciled safely. AWS's guidance on idempotent APIs illustrates why repeating a request can otherwise create unintended duplicate work. The product requirement is a reliable, understandable recovery path, not an unqualified exactly-once guarantee. [8]

Adopted default: Add one bounded external action only after a pilot establishes that the action is necessary to finish a valuable recurring workflow. Retain human control for the relevant approvals. Longer automation chains and scheduled execution follow explicit scope decisions and evidence of usefulness.

For example, a future order-exception app could compare orders with shipments, present the discrepancy, record an owner's decision, perform one approved follow-up, and verify its result. That is a defined workflow expansion. It does not imply that Forma has become a complete order-management system.

11 Collaboration sharing and independent reuse

Forma supports collaboration through persistent work with a known audience. Colleagues should be able to see the permitted evidence, active context, findings, decisions, and remaining questions without reconstructing the author's private conversation with AI.

The product distinguishes three sharing operations at the point of use.

Sharing operation	What is shared	What remains independently controlled
Share an app	A reusable tool and its published behavior.	The recipient's data, work context, and current access.
Share saved work	A particular case, review, or board with its permitted content and state.	Audience, edit rights, source access, and sensitive content.
Share an export	A selected snapshot or handoff for a stated audience.	Export permission, included fields, and subsequent distribution.

Reusing an app must not silently copy the author's evidence, private notes, credentials, or privileges. The next person selects a new case or work item and compatible data. Forma checks field meanings and current access. Missing inputs or permissions produce an explanation rather than an apparent successful result.

Sharing the same case intentionally allows colleagues to work on that case, within the permissions granted. App access and data access remain distinct. A person may be allowed to use Outbound Investigation without being allowed to read Maya's particular evidence.

Adopted default: Committed records, findings, and decisions in shared work are visible to its authorized audience, which stays apparent. Personal drafts are clearly marked and remain private until deliberately added. The first release supports saved handoff and change history, with clear handling when someone edits a stale version. It does not promise live simultaneous coediting.

An export identifies its source context, relevant version, filters, and time where needed for interpretation. Revocation prevents future authorized access through Forma; it cannot recall information someone already saw, copied, or exported. The sharing experience should make the audience and content concrete before distribution.

12 Publication ownership and application lifecycle

The visible lifecycle is Draft, In review, Published, and Retired. A published app can have a separate draft. People using the current approved version should not encounter unfinished changes from its author.

Review focuses on what materially changes: data sources and fields, interpretations, calculations, joins, interactions, external actions, and requested access. The reviewer sees a concise difference summary, representative results, and a route to inspect details. The product should reduce repeated review of unchanged approved behavior.

Adopted default: New shared apps and material changes follow the organization's publication policy. Routine filtering, personal arrangement, and ordinary notes within approved behavior do not trigger app publication review. The Northstar demonstration uses Sarah as reviewer; other organizations can assign appropriate responsibilities without reproducing those fictional personas.

Published versions have a stable identity and an owner. New work starts with the current permitted published version. Existing saved work records the version used. When a newer version is available, the app explains the relevant changes and an authorized user explicitly chooses whether to apply a compatible update to that saved work. Material calculation changes must not silently rewrite the meaning of earlier findings.

Rollback restores an available, compatible version and preserves history. It does not restore revoked permission or erase intervening work. A critical security restriction can prevent further use of a previously published version even if a case was saved with it.

Owners maintain descriptions, input requirements, dependencies, and useful defaults. When a source changes, the app identifies affected work and the need for repair. A departing builder should not leave the company with an ownerless tool. Administration includes transferring responsibility and retiring obsolete apps.

Retirement removes an app from recommended new work while retaining an understandable historical record according to retention policy. Deleting work, deleting a source revision, and retiring an app are separate actions with different consequences. The product should explain those consequences and avoid silently breaking referenced findings.

13 Secure cloud organization access and data handling

For this product, secure cloud means that Forma operates the hosted application service and provides clear controls over company membership, app publication, data use, and permitted actions. It also means taking responsibility for service operation, recovery, ownership, and understandable failure states.

The managed offering hosts company accounts, the apps and their runtime experience, and the work the service promises to preserve. It can use existing identity services. The product does not require inventing a new identity provider or copying every connected database into Forma.

Companies and tenancy

The intended service supports multiple organizations with separate membership, app libraries, sources, policies, and work. Shared product infrastructure must not imply shared customer data. Whether particular resources are shared or dedicated belongs in the technical design and deployment offering.

A single-organization pilot can demonstrate a useful workflow. Claiming that the service serves multiple companies securely requires verification of those boundaries, including search, cached results, files, background work, and administration. A themed space is not a substitute for organization separation.

The default is one managed service. Dedicated environments, customer-hosted arrangements, and additional regional offerings are later commercial choices when actual demand supports their cost. The product description should not promise all deployment modes at launch.

Access throughout use

Discovering an app, running it, viewing its data, editing its definition, reviewing publication, and exporting results are different permissions. The service evaluates current authority when work is opened and when data or actions are requested. A copied link, saved layout, or slash command does not grant authority.

AI can propose a change but cannot approve its own broader access. Sensitive operations require the organization's applicable authorization. A visible denial should state the practical limitation and allowed next step without exposing protected data.

Imported files, logs, and external text can contain instructions intended to influence a model. OWASP describes this indirect prompt-injection risk and notes that retrieval alone does not remove it. Forma's

product requirements therefore keep evidence separate from authority and test specific prohibited operations. [7]

We will demonstrate the controls actually implemented and tested. A version hash identifies particular content; it does not prove correct logic or universal safety. Runtime restrictions contribute to protection, while source authorization, content admission, operational controls, and review remain application responsibilities.

Data handling and service expectations

Customers need to understand what is stored, what is sent to a model, where relevant processing occurs, how long work and sources remain available, and how deletion affects history. Generated summaries and search indexes can contain sensitive information too. Policies apply to them as well as to original files.

The first offering needs clear save behavior, recovery expectations, support ownership, and usage limits. It must explain whether a result is current, stale, partially loaded, or unavailable. Formal service commitments follow operational evidence rather than an unsupported availability promise.

Government and regulated industries remain possible future customers. Selecting a cloud provider or using encryption does not establish Forma's own compliance status. Any FedRAMP claim must refer to an actual offering and its verified status; current FedRAMP guidance also distinguishes the cloud service certification from an agency's authorization of its use. [9]

14 AI APIs and MCP in the product

AI interprets intent, suggests data mappings, finds or adapts tools, creates supported missing behavior, and helps draft explanations. People should be able to see and correct consequential choices. An AI explanation must be traceable to evidence when it makes claims about company data.

APIs and connectors provide the operations a source system exposes. MCP is another integration mechanism for tools and context. The official protocol describes tools, resources, and prompts; it does not define every application's business rules or replace the host's product responsibilities. An app does not need both a separate API integration and MCP integration for every source. [5]

The platform adds the shared experience around those ingredients: identity, app ownership, persistent work, defined calculations, publication, permitted actions, and reliable use. Generative UI gives the task an adaptable interaction layer within that product.

Model policy and predictable use

The first release supports a chosen generation provider under an explicit organization data policy. Where schemas and synthetic examples are enough, generation should use them. Sending sensitive records to a model requires a clear supported policy and customer understanding of the actual provider arrangement.

Additional providers or customer-controlled model accounts can follow demand. The interface should make the selected provider, relevant data handling, and usage visible. A provider selection alone is not proof of a particular residency, retention, or compliance guarantee.

Opening an approved app, navigating spaces, filtering supported data, and recording ordinary work should not require regenerating the application. If generation is unavailable, existing apps should

continue where their other dependencies permit. Apps that genuinely require a model for a specific operation must show that dependency and its failure state.

Generation, storage, queries, and support have costs even when an existing component is reused. Customers need understandable usage limits and feedback before an expensive or long-running operation. Public performance and cost claims will follow measurements of the supported product.

Assistant surfaces

The browser workspace is the initial home. Later, a selected app or view may be available within a supported assistant through MCP Apps. The official extension supports interactive interfaces inside compatible hosts, with host-specific integration and capabilities. [6]

The goal is to reuse the approved app behavior and data rules across surfaces where practical. A shared resource address alone does not make the app portable with zero work. Each supported host must be validated for identity, permitted data, interactions, state, and its actual sandbox behavior. A public website builder is not part of the first platform release.

15 Why Forma Stack matters and what is established

Forma Stack 3.0.0 is the foundation reference for this product. Its architecture describes a compiler, Rust rendering, browser signals and islands, and an explicit HTML runtime. Its runtime security model includes owned roots, declared tiers, and grants for specific capabilities. These are relevant foundations for composed interfaces and controlled behavior. [2][3]

They support the reason for building Forma: treating applications as structured, reusable software that can be composed and changed deliberately. The platform applies that foundation to people creating and sharing small tools over company data. Its business value will depend on whether the complete experience reduces the effort of creation, review, use, and maintenance.

The security policies also state responsibilities and limits. Mounting constraints are not a universal HTML sanitizer. The hosting application must admit content and assign appropriate privileges. The reviewed 3.0 code treats compiled FMIR modules as trusted application-build output; general admission of arbitrary untrusted modules is not supplied by that boundary. [3][4]

Area	What is established for this definition
Forma Stack foundation	Selected 3.0 documentation and source policies support the framework capabilities described here.
Security and navigation prototypes	Existing demonstrations illustrate the narrative and interactions; fixture behavior is not proof of production enforcement.
Full company platform	Organization administration, app hosting, data workflows, lifecycle, and independent reuse must be demonstrated together.
Performance and security outcomes	Require product-level measurement and validation on supported workloads and boundaries.

The review of the foundation does not establish that the complete canvas, ingestion service, vector registry, application review process, or multi-organization operations are finished. Some of that work may exist elsewhere. Its readiness must be stated from the relevant application evidence, not inferred from the framework version number.

Another team could build a comparable customer experience with different technology. Forma's stack becomes an advantage when it makes useful apps easier to create, constrain, adapt, and maintain. The pitch should connect technical work to those outcomes and avoid claiming that a language or runtime alone makes competitors incapable.

Founder explanation: I built Forma around structured, reusable applications and precise changes. The platform gives that foundation a practical purpose: teams can form apps around their data and work, run and share those apps, and transform them as their needs change. Northstar illustrates one use of the platform; the product is the common environment in which teams create and maintain their own tools.

16 The platform story and six act demonstration

The main story follows a company using Forma to create and maintain its own apps. The audience should see the company home, creation experience, running apps, shared work, and growing app collection. Northstar can supply concrete names and sample data, while the demonstrated product remains Forma.

Act 1 Enter the company workspace

A team member opens the company home and sees its permitted spaces, apps, and recent work. The person can create a space, start an app, or resume work according to their role. Slash commands and clicks provide equivalent navigation. The platform is visible before the audience enters a particular example app.

Act 2 Bring data and describe the job

The person uploads a supported spreadsheet or log file, or selects a supported connection. Forma inspects the data and makes important mappings and limitations visible. The person describes the task, desired result, and relevant rules. Importing data does not automatically publish it to the company.

For the Northstar example, Maya selects a Security space and brings connection events and a destination inventory. Another example can use spending records or task data. The source-specific meanings are supplied within the same platform workflow.

Act 3 Form and transform a working app

Forma retrieves suitable permitted components, adapts compatible behavior, and generates supported missing pieces. The result is a useful draft with real interactions and understandable calculations. The person refines it through focused requests or direct controls and can preview, accept, undo, or discard changes.

Show a meaningful transformation: add a comparison, introduce a supported input form, or change a correctly defined measure. Preserve relevant work while making the effect visible. A new color or title alone does not demonstrate the ability to create a different working app.

Act 4 Publish and run the app

The app receives the review appropriate to its material behavior and audience, then becomes a known published version in the company library. The team can run it as an ongoing tool, save its work, and return later. The app's owner and required inputs remain clear.

The Northstar version uses Sarah to illustrate review. Other teams can assign the responsibility differently. Publication shares the reusable app with the intended audience; it does not silently publish the author's private data or credentials.

Act 5 Share work and reuse the tool

A colleague opens permitted shared work and understands its context, records, decisions, and remaining questions. Another person can start new work using the same app with compatible data and independently evaluated access. These are distinct paths in the interface.

In Northstar, Jordan starts a new case for ops-runner-04 using the app Maya created for build-worker-17. This demonstrates that the first person's tool can serve another person's task without copying private case content or regenerating the complete interface.

Act 6 Build a different app in the same platform

The team returns to the company workspace and creates a different supported app through the same authoring path. A compact example could turn a spending table into a review queue with a detail view and an internal decision form. It uses the same data interpretation, generation, publication, access, and saved-work capabilities.

The new app can live in another space. Typing `/finance` or clicking Finance reveals that app and context; returning to Security preserves the earlier work. The demonstration must show a meaningful difference in purpose and behavior, not merely two fixed dashboards behind different colors.

This final act establishes the launch product: Forma lets teams build a collection of their own apps. The examples should be reproducible through supported platform capabilities without custom platform code for each example. Fixture data or simulated behavior is labeled wherever used.

17 Example apps and the creator platform analogy

Teams can create different apps using the common platform capabilities. Each app still requires appropriate data meaning and workflow rules. A new space alone does not create domain expertise. The examples below illustrate possible outputs within supported capabilities; they are not separate industry products, customer claims, or a promise that every specialized behavior is available at launch.

Space or team	Example inputs	Example app	Domain work still required
Security	Connection events and asset inventory	Outbound investigation	Time windows, match rules, evidence interpretation
Finance operations	Invoices and payment records	Reconciliation and exceptions	Currency, matching, tolerances, review authority
Operations	Orders, shipments, and stock records	Fulfillment exception review	Status meaning, expected dates, ownership
Support	Tickets and account context	Escalation workspace	Priority rules, customer context, permitted follow-up
Healthcare operations	Staffing and capacity schedules	Coverage review	Role definitions, coverage requirements, sensitive data handling

The Northstar reference app

Northstar is a fictional company used in the existing security demonstration. Maya creates an investigation tool, Sarah reviews its shared version, and Jordan reuses it with different evidence. That story remains a useful reference scenario and potential starter template built on Forma.

Card	Job in the example investigation
Case summary	Show the asset, question, window, source revisions, and defined metrics.
Activity timeline	Show when activity occurred and link selections to underlying records.
Destination breakdown	Compare observed destinations and transfer measures.
Inventory comparison	Explain supported matches, missing entries, and reference limitations.
Evidence table	Inspect, filter, and reference permitted source records.
Findings and handoff	Preserve observations, supporting evidence, uncertainty, and next steps.

The app consumes existing evidence. An alert, large transfer, or destination absent from an inventory does not establish compromise. Statistical anomaly comparisons require sufficient history and a defined baseline. Findings explain their evidence, match rules, and limitations.

The demonstration can show a specifically unauthorized operation being refused, including a request influenced by an attacker-controlled log field. It must identify whether the enforcement is real or simulated. That example illustrates one control and does not establish universal protection against prompt injection.

The healthcare example concerns operational coordination. Clinical decision support, medical diagnosis, and other specialized uses require their own validated product scope. Government use similarly follows actual customer, deployment, and applicable requirements.

The common building blocks include source inspection, tables, comparisons, timelines, filters, notes, review, publication, and reuse. The variable parts include business definitions, supported integrations, calculations, actions, and the evidence required to trust the result.

What the Roblox analogy contributes

The useful analogy is a creator platform. Forma supplies the environment to create an app, run it, discover it, and reuse it. An organization's app library is the place its people find their work tools. Reusable components and templates make subsequent creation easier.

For the initial product, each company has its own permitted app catalog. There is no automatic sharing of company data across organizations. A future template exchange would distribute selected app definitions or starter components separately from evidence, credentials, and access.

The analogy is a way to explain the product structure. It does not establish a marketplace, network effect, or a particular economic outcome. Lead with the customer's working experience; use the analogy when it helps the audience understand the broader platform ambition.

18 Release scope and product completeness

The first release is the Forma platform. Its scope is defined by the app capabilities people can use, rather than by an industry. Real users must be able to form and adapt different supported apps, publish them, perform work, and let colleagues continue or reuse them with appropriate data and access.

First usable platform

The initial scope includes a company account and authenticated membership, a private app catalog, company-defined spaces, and both slash and click navigation. It includes clear app ownership and the ability to continue existing work or start a new instance.

The working experience includes a saved canvas, CSV and bounded XLSX import, visible data interpretation, reusable views and forms, permitted discovery, and meaningful generation within supported capabilities. Users can refine an app, inspect calculations, undo proposed changes, and maintain internal records and source-linked decisions.

Sharing includes app publication, saved-work handoff, explicit export, independent access evaluation, and a known application version. The product distinguishes personal display changes from material shared-app changes. Denials, source problems, missing inputs, and save failures are understandable.

Connected source access is initially read-only. Supported internal forms, records, notes, statuses, and decisions remain writable under the app's rules. Multiple apps must coexist with their own purposes, inputs, versions, and saved work. The hosted service preserves that work and meets the responsibilities stated in Section 13.

First extensions after demonstrated use

Add connectors, reusable components, or workflow capabilities selected from repeated customer needs. Introduce a bounded external action when it completes a real task the current platform cannot finish. Refine administration and usage visibility from operating the service. These additions broaden an existing platform; they do not mark a later transition from a security app into a platform.

A selected MCP Apps host is a separate extension after the browser workflow is dependable. Its priority follows user demand. More model choices and customer-controlled model accounts should solve concrete customer requirements.

Later possibilities

Live simultaneous editing, scheduled workflows, broader action integrations, optional starter packs, dedicated deployments, and cross-company template distribution remain possible expansions. A public marketplace is a later choice. None is a condition for proving the first useful private company platform.

The initial platform supports a defined range of app behavior rather than arbitrary code or every business operation. Specialized systems such as packet capture, clinical diagnosis, autonomous remediation, or a full accounting engine require additional capabilities and validation. Their appearance in a possible customer workflow does not make them implicit platform commitments.

Evidence of product completeness

Before describing the platform as ready for its supported use, demonstrate that a user can create an app beyond the supplied demo without a developer changing platform code for that app. At least two

apps with different purposes or interactions should exercise the same creation, publication, access, and saved-work model. This tests platform generality without requiring two complete industry products.

Also demonstrate that a second user can complete supported work independently, saved work reopens with its context intact, revoked permission prevents further access, and a changed source or calculation produces an understandable state.

Also demonstrate that a published app remains usable when a new draft or generation attempt fails; that a reused app binds to the new user's permitted data; and that organization boundaries hold wherever the offering claims to serve multiple companies. These are customer outcomes and trust requirements, not a substitute for the separate engineering validation plan.

19 Adoption measurement and the commercial approach

The initial commercial hypothesis is a managed platform subscription for a team or organization, with understandable included usage and limits. Exact prices and billable units remain open. Earlier illustrative customer names, prices, and revenue scenarios are not traction.

Begin with a bounded pilot around a repeated real workflow. Establish how the team currently performs the task, who does the preparation and review, and what makes the result useful. Compare Forma across the whole process, including cleanup, creation, publication, investigation, handoff, and the next reuse.

Measure	What it should establish
Time to a useful approved app	Whether creation and review together improve the process.
Preparation and cleanup effort	Whether import and interpretation save work overall.
Correctness and traceability	Whether calculations and findings retain their supporting evidence.
Second-person reuse	Whether an app survives beyond its author's original case.
Repeat use without founder operation	Whether the product is understandable and independently valuable.
Maintenance and reviewer effort	Whether useful apps remain affordable to keep working.
Service cost per completed workflow	Whether generation, queries, storage, and support costs permit a viable offering.
Willingness to pay and continued use	Whether the value supports a commercial relationship.

Measure reuse across later work, not merely repeated loading of the same fixture. Measure perceived speed from the request to a usable result, not only a narrow retrieval or DOM operation. Include relevant file sizes, data conditions, and concurrency when publishing performance results.

Evaluate whether teams can create and maintain more than one useful app through the same platform. If each app requires a custom platform rebuild or continuous founder operation, the creation promise is not yet established. A pilot should also determine whether the customer's main problem is creating and operating tools or something a Forma app cannot supply, such as unavailable source data.

Pricing should reinforce adoption of useful apps while accounting for service cost. An organization subscription with bounded usage is a sensible starting hypothesis. The final commercial unit should follow the buyer's value, procurement needs, and observed workload rather than a predetermined price copied from an earlier example.

20 Positioning the YC connection and the explanation to use

Lead with the platform's customer job, then use one concrete app to demonstrate it. The company's scope can be horizontal while its first sales conversations and demo remain specific.

The verified YC request

The relevant request is **A Cloud for Small Software**, by **Pete Koomen**, in YC's **Fall 2026** Requests for Startups. It discusses the difficulty of deploying and sharing small team tools, including authentication, permissions, company customization, and secure distribution. Its central line is: "Small software should be as easy to share with your colleagues as a Google Doc." [1]

Our interpretation is that Forma aligns with that problem by giving company apps a place to run and be shared with maintained access and work state. The request does not prescribe a security-only app, and it is not a promise to fund Forma. Customer evidence and a working product still determine the strength of the business case.

Platform explanation

Forma is a platform where teams build, run, and share AI-generated apps over their company data. Bring in a spreadsheet or connect a supported system, describe the task, and create a working app. Your team can use it in a shared company space, keep the decisions and evidence, and reuse it for the next job.

Short customer explanation

Forma is where your company's small apps live. Your team can create a tool for the way it works, publish it to the right people, and return to the same app and saved work. Security, finance, and operations can have their own spaces within the same platform.

Demonstration explanation

The Northstar demo shows a security team using Forma to create and reuse an investigation app. Maya forms the tool, Sarah reviews its shared version, and Jordan uses it with another case's data and permissions. A different team can use the same platform capabilities to create another supported app for its own work.

The reason for building the framework

I built Forma to make software structured, reusable, and easier to change precisely. The platform uses that foundation to help teams create small apps that can be run, reviewed, and shared over company data. The outcome we need to prove is less effort to create and maintain useful tools, with clear behavior and appropriate access.

The claims that earn the position

The proposed distinction is the complete workflow: data interpretation, adaptable interaction, durable work, reviewable app changes, and independent reuse in one hosted environment. We need to demonstrate those benefits against the customer's current process. Assertions about competitors' internal stacks or universal inability to handle data do not establish the advantage.

Describe the live product separately from the vision. Show what is implemented, label fixture or simulated behavior, state measured supported limits, and identify the next capability under development. The platform ambition should remain clear without presenting every planned extension as a current feature.

21 Reference notes

External references were reviewed on October 22, 2026. The Forma foundation references below identify the specific 3.0 revision used for this explanation. They support the stated framework properties and limits; they are not a product-level benchmark or audit of the completed cloud service.

[1] Y Combinator Requests for Startups. Fall 2026, A Cloud for Small Software by Pete Koomen. The official page contains the request and its author. Open reference

[2] Forma 3.0 architecture. Foundation packages, rendering, browser behavior, and explicit HTML runtime at commit 2006423b21fe0f8dd5a510bd09bfe3b8748327a5. Open reference

[3] Forma core security policy at the same revision. Explicit mount ownership, grants, raw markup responsibilities, and stated limits. Open reference

[4] Forma Stack security policy at the same revision. Trusted FMIR input and the boundary around untrusted module admission. Open reference

[5] Model Context Protocol architecture overview. Official description of hosts, tools, resources, prompts, and protocol scope. Open reference

[6] Model Context Protocol MCP Apps overview. Interactive interfaces, host integration, and host-dependent support. Open reference

[7] OWASP Gen AI Security Project. Prompt Injection guidance, including indirect instructions in external content and the limits of retrieval as a mitigation. Open reference

[8] Amazon Builders Library. Making retries safe with idempotent APIs, explaining duplicate side effects and retry semantics. Open reference

[9] FedRAMP Consolidated Rules for 2026. Using a FedRAMP Certified Cloud Service, including the distinction between service certification and agency authorization. Open reference

22 Remaining open decisions

Forma is the launch product. Company spaces, navigation, the shared-app model, and forming and transforming apps are settled. Northstar remains a demonstration. The following choices require customer or operating evidence; their defaults allow work to proceed.

First paying customer and buyer

Default: Select a team with recurring custom-tool needs, usable data, a champion, a sponsor, and at least two users. **Resolve through:** observed work, independent app use, and willingness to pay. Existing relationships can help recruitment; the demonstration's industry does not determine the first buyer.

First live connection

Default: Retain file import and add one source repeatedly needed by pilots. **Resolve through:** frequency of use, permission and API feasibility, data meaning, maintenance burden, and time saved. The first release does not require a broad integration catalog.

First external action

Default: Launch with source reads and internal work records. Add one bounded action when a valuable task otherwise remains incomplete. **Resolve through:** its intended effect, authorized actor, supported source behavior, and recovery path. Connecting an API does not authorize this expansion.

Model provider and sensitive data policy

Default: Use one supported provider, minimize record exposure, and state the data policy. **Resolve through:** quality, cost, buyer requirements, provider terms, regional processing, and permitted use of sensitive content. Customer-controlled model accounts follow demonstrated demand.

Service limits and operating commitments

Default: Publish only measured supported file, row, query, concurrency, and usage limits. **Resolve through:** representative workloads, failure recovery, cost, and actual operating evidence. Set appropriate retention and support commitments for the first customer agreement rather than inherit unverified timing or scale claims.

Pricing and billable unit

Default: Test a team or organization subscription with clear included usage. **Resolve through:** repeated customer value, active use, service costs, support effort, and purchasing behavior. Do not present an illustrative price as a validated willingness to pay.

Next reusable capability or customer workflow

Default: Prioritize a capability needed across real customer-created apps. Use one rich demo and a compact different example to test generality. **Resolve through:** repeated demand, expected value, and whether the same authoring path supports the work. Do not prescribe an industry sequence from the Northstar example.

Additional deployment and regulated market requirements

Default: Offer one managed service with clear company separation and an accurately stated data policy. **Resolve through:** qualified customer needs for dedicated resources, geography, contractual controls, or a particular regulated offering, together with the cost of operating it. No compliance status is assumed.

Priority of live coediting and assistant hosts

Default: Deliver persistent shared work in the browser first. **Resolve through:** whether simultaneous editing or a particular assistant is essential to recurring work, plus validated host behavior. Choose the first extension from that evidence.

Cross company templates and a public marketplace

Default: Keep organization catalogs private. **Resolve through:** template or partner demand, content rights, review and support responsibilities, and separation from company evidence and credentials. A marketplace remains a later business decision.